Lecture plan · for the lecturer

Attention and transformers

AI · University · Lecture 3

Time	Materials
60 min	Projector or board; The lecture slides and the seminar sheet from the kit; Phones or laptops with Darsi, for the lab

Learning goals

- Compute scaled dot-product attention.
- Explain √d and position.
- Sequence the transformer for novices.

Activity: One head by hand

1. Warm-up (5 min): Show the sentence “The student couldn’t fit the laptop in the bag because it was full” and ask students how they resolved “it”.
2. Compute (20 min): Pairs compute QK^T, scale, softmax and the weighted values for three 2-D tokens.
3. Break (10 min): Repeat with d = 512 random vectors without scaling and look at the weights.
4. Build (20 min): Add heads, a residual path and positions, each with the problem it solves.
5. Wrap-up (5 min): Students write the muddiest point of the lecture and one question; read them before next week.

Discussion

- What happened to the weights without √d?
- Can attention maps explain a prediction?
- Which part will you teach last?

Exit ticket

- Write $\text{softmax}(QK^T/\sqrt{d})V$ and label what each letter means.

Try it in your next lecture

Give pairs 2-D query, key and value vectors for three tokens; they compute scores, softmax and output by hand, then compare with three lines of NumPy.

Seminar sheet · for students

Attention and transformers

AI · University · Lecture 3

Name _____ Group _____ Date _____

1 **Attention alone treats inputs as a set. What makes word order matter?**

◯ A larger vocabulary ◯ More attention heads ◯ Positional encodings

2 **Compute one attention row. A query scores the three keys 2, 0 and 0 after scaling.**

With $e^2 \approx 7.4$, the softmax weight on the first key is about _____ %, and on each of the others about _____ %.

Word bank: 33 · 79 · 11 · 50

3 **Which statements about self-attention are true? Choose all that apply.**

☐ Each head's weights in a row sum to 1 ☐ Its cost grows with the square of sequence length

☐ It needs recurrence to see distant tokens ☐ Without position information it ignores word order

4 **Dividing by $\sqrt{d}$ is mainly there to make the computation faster.**

◯ True ◯ False

5 **Match each component to its job.**

Positional encoding	• • • •	Capture different relations
Multiple heads	• • • •	Stops a token seeing future tokens
Residual connection	• • • •	Injects word order
Causal mask	• • • •	Keeps gradients flowing in deep stacks

Answers · for the lecturer

Attention and transformers

AI · University · Lecture 3

1 **Positional encodings**

Position information is added to each token's embedding.

2 **With $e^2 \approx 7.4$, the softmax weight on the first key is about 79 %, and on each of the others about 11 %.**

7.4 / (7.4 + 1 + 1) ≈ 0.79, and 1 / 9.4 ≈ 0.11 each.

3 **Its cost grows with the square of sequence length; Without position information it ignores word order; Each head's weights in a row sum to 1**

Every token attends to every token directly, so no recurrence is needed; softmax rows sum to 1; cost is $O(n^2)$.

4 **False**

It keeps score variance near 1 so softmax does not saturate and gradients keep flowing.

5 **Positional encoding — Injects word order**
Multiple heads — Capture different relations
Residual connection — Keeps gradients flowing in deep stacks
Causal mask — Stops a token seeing future tokens

Attention alone is blind to order, so positional encodings add it; several heads capture different relations; residual paths give gradients a direct route through deep stacks; and the causal mask hides later tokens, so the model cannot peek at the word it must predict.

For you to think about

What is the minimum a student must compute by hand before using a transformer library in your course?